

Design Brief

Summary

Develop a 64bit Windows desktop application using C/C++ that implements the set of features defined in this document. Submit your application's source code in a form that can be built and run by our review team.

Requirements

The application needs to present 4 viewports concurrently in a single window. The contents presented in each viewport should be read from each of the 4 .dat files that we have provided with this document. To clarify, the contents of v0.dat should be displayed in one viewport, the contents of v1.dat should be displayed in another viewport, etc. Each .dat file contains a sequence of .bmp files.

Each viewport will be required to cycle through and present each of the bitmaps one by one in its designated area, in the order the bitmaps are defined in the file. The amount of time to wait before cycling to the next bitmap is defined in the header of each .dat file. Once a viewport has completed presenting all the bitmaps in its .dat file, it should go back to the beginning of the bitmaps and start again, and continue cycling endlessly.

When the user presses the F5 key at a time that the window has focus, all the viewports are to concurrently restart their cycling from the beginning of the content within their respective .dat files.

The following behavioural attributes should be observed:

- There should be no delays experienced when pressing F5 and seeing the first bitmap presented in each viewport
- There should not be any delay when closing the window and terminating the application
- There should not be any noticeable flickering seen during transitions within a viewport

A video of a minimal completed implementation is provided with this document.

Start-up Project

A Visual Studio 2022 start-up project and solution has been provided which contains a basic main function that creates and launches a window for you. You may use this start up project (change, extend, etc.), or you are free to create your own from scratch. There is a header file called `FileStructures.h` that is included in the start-up project, which contains the definition of the structures used in the .dat files (see later section for more info on .dat files).

Build/Runtime Dependencies

To reduce the risk of the reviewing team encountering issues trying to build and run your application, you should look to keep the amount of non-Windows SDK dependencies to a minimum. Ideally, there would be zero 3rd party/open-source libraries to be concerned with. If you do choose to make use of dependencies that are not part of the Windows SDK, you will need to provide their headers and libs (and DLLs if required) with your submission. The reviewing team has a very strong preference for the use of vcpkg to manage open-source

dependencies, so it will be acceptable to enable vcpkg for your project, enabled manifests, and just provide the manifest with your submission.

If you choose not to provide a Visual Studio project file with your submission, please ensure you provide some other kind of makefile, CMakeLists.txt, etc.

The .dat File structure

The .dat files provided are simply a sequence of .bmp files delimited by some file structures. The struct packing alignment settings have not been changed from MSVC defaults, and the files were created on a AMD64 architecture CPU.

File Header

Each file starts with a `DatFileHeader` structure. The file header contains a signature sequence of chars, a value describing how many images are in the file, and a value for the duration of time each image is to be shown in the viewport for the .dat file.

```
struct DatFileHeader
{
    char Signature[sizeof(FILE_HEADER_TAG)];
    unsigned short NumImages; // The number of images in the file
    unsigned short CycleTimeMs; // Cycle duration in milliseconds
};
```

DatFileHeader

struct DatFileHeader		
Offset	Field name	Size
0	char DatFileHeader::Signature[5]	5 bytes
6	unsigned short DatFileHeader::NumImages	2 bytes
8	unsigned short DatFileHeader::CycleTimeMs	2 bytes

struct DatFileHeader		
	0 bytes	
0	char DatFileHeader::...	
2		
4		
6	unsigned short Dat...	
8	unsigned short Dat...	

File Content Sequences

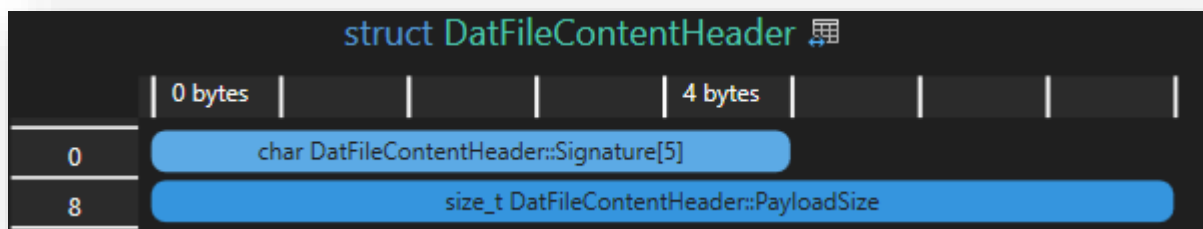
Following the file header is the content of the file. A slice of content represents one single .bmp file preceded with a `DatFileContentHeader` structure and proceeded by a `DatFileContentTrailer` structure. There are one or more slices of content within each .dat file.

Note: The payload of each content slice contains a .bmp file, so the appropriate headers of a .bmp file will be present.

```
struct DatFileContentHeader
{
    char Signature[sizeof(CONTENT_HEADER_TAG)];
    size_t PayloadSize; // The size of the proceeding .bmp file
};

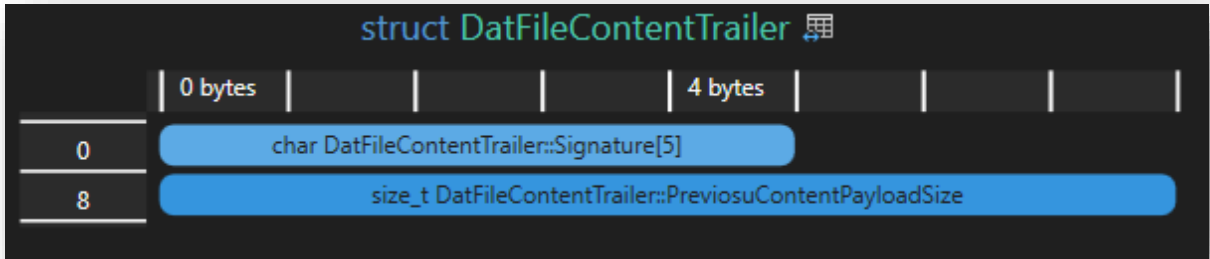
struct DatFileContentTrailer
{
    char Signature[sizeof(CONTENT_TRAILER_TAG)];
    size_t PreviousContentPayloadSize; // The size of the preceding .bmp
};
```

DatFileContentHeader



struct DatFileContentHeader		
Offset	Field name	Size
0	char DatFileContentHeader::Signature[5]	5 bytes
8	size_t DatFileContentHeader::PayloadSize	8 bytes

DatFileContentTrailer



struct DatFileContentTrailer		
Offset	Field name	Size
0	char DatFileContentTrailer::Signature[5]	5 bytes
8	size_t DatFileContentTrailer::PreviousContentPayloadSize	8 bytes

Example .dat file

Below is an example of the structure of a .dat file containing 2 content slices

